Budget Together

PROJECT SPECIFICATION

1. Project description

A responsive budgeting web app with automatic US bank imports, private and shared budgets, category balances, and monthly savings targets. The design uses original layouts with Monarch as a reference for clarity. The service is intended to operate at cost; pricing depends on operating and connectivity expenses.

2. Target users and purpose

Individuals track income, spending, and remaining funds. Couples manage shared finances while keeping personal accounts private. Both can assess savings and plan the next month.

3. Core budgeting rules

Access: Each user has a private budget and may share one budget with one partner. Each account belongs to one budget. Partners equally control shared categories, limits, classifications, and savings targets; private data stays owner-only.

Zero-sum plan: Expected take-home income + incoming shared contributions + explicit carry-forward = category limits + savings target + outgoing shared contributions. A contribution reduces private funding and increases shared funding equally. Drafts may be incomplete; activation requires an exact balance. Limits and savings targets cannot be negative.

Limits: Category limits repeat until changed; current-month edits must preserve balance. Actual income is tracked separately, with differences shown for plan adjustments. Overspending shows a negative balance and alert without blocking purchases or changing other limits.

Savings: Actual income + actual incoming contributions - actual outgoing contributions - actual spending. Same-budget transfers and credit card payments for purchases already counted do not affect savings. Each budget has one monthly target; the app tracks monthly results, not cumulative balances, and moves no money.

Leftovers: No automatic rollover. Users may explicitly fund next month's categories or savings, accounting for overspending and income shortfalls. Carry-forward is a separate funding source; retained savings and reserved funds stay distinct. Historical income, spending, savings results, and targets stay unchanged, with no extra savings transaction or duplicate allocation.

4. MVP features

Registration, login/logout, password reset, and profiles; automatic checking, savings, and credit card imports; private/shared account assignment; accepted email invitations; balanced monthly plans; suggested income/purchase classifications; custom categories; transaction search, filters, notes, recategorization, splits, and exclusions; optional manual entries; category dashboards; monthly reviews; mobile and desktop support. Categories and manual transactions support full CRUD; category deletion requires reassignment. Automatic bank connectivity is required.

5. Additional features

In-app alerts at 80% and above 100% of category limits; recurring-charge suggestions with estimated amount/date and confirm/dismiss controls; historical category and total-spending graphs; immediate shared updates without refresh. Real-time collaboration fulfills the advanced-feature requirement.

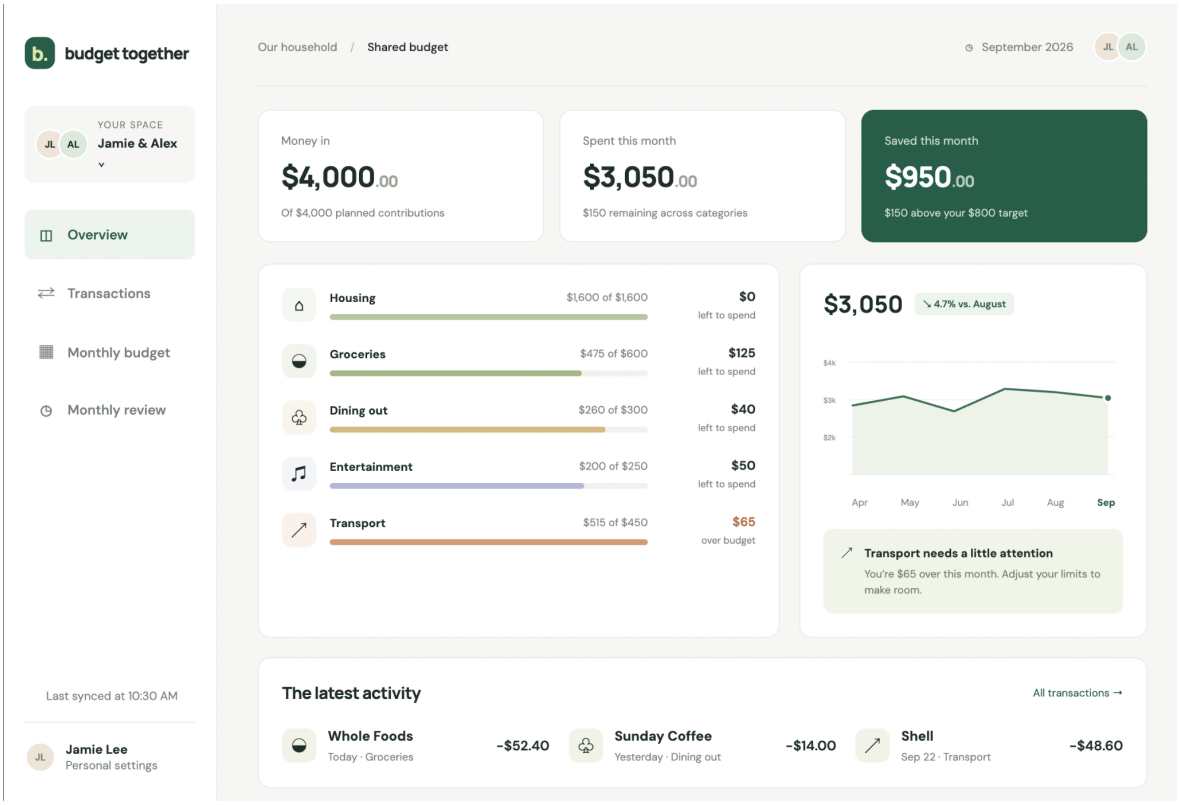
6. Main user flows

Setup: Register, connect accounts privately, optionally invite a partner and share selected accounts, enter income/contributions, allocate funds, and activate.
Daily check: Select a budget, check category balances, and open recent transactions.
Month end: Compare results, correct classifications, allocate leftovers, and balance next month. Reopening a review updates existing allocations without duplicating funds.

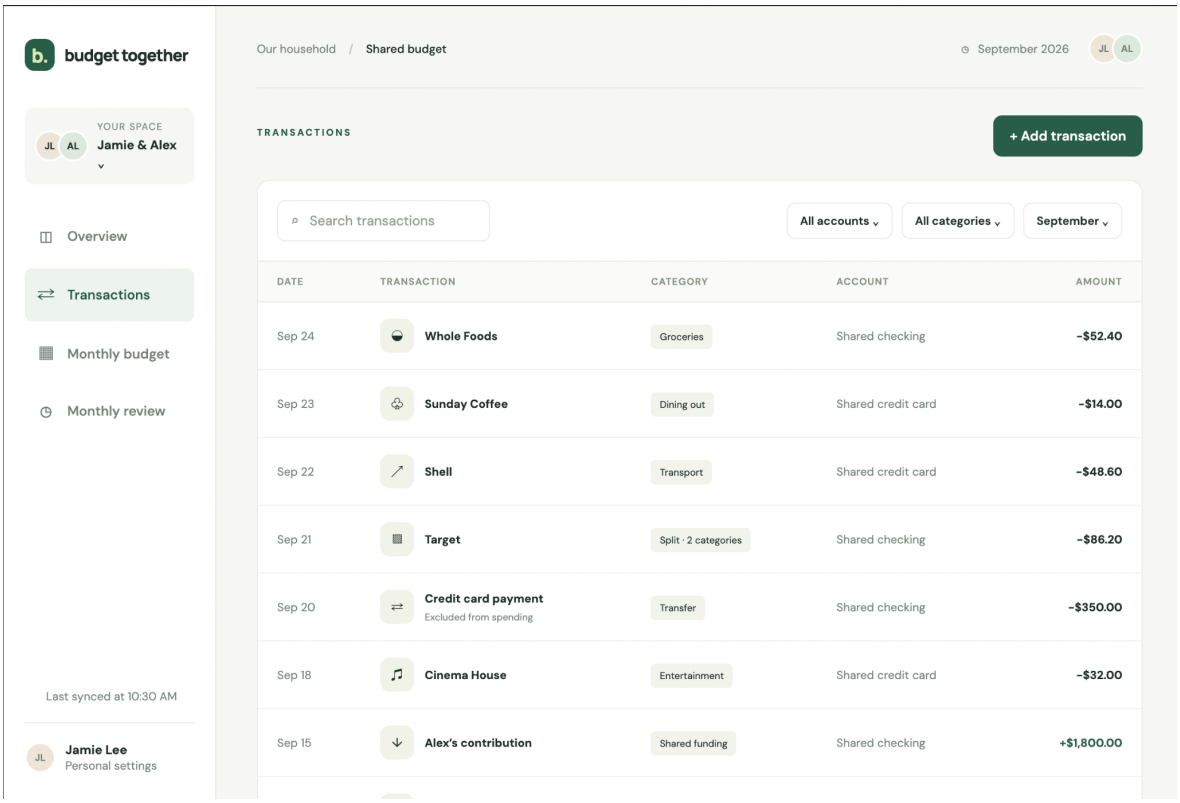
7. Views and wireframes

Core views: authentication, onboarding, dashboard, transactions, budget editing, monthly review, and accounts/settings. Original wireframes are retained below.

Dashboard

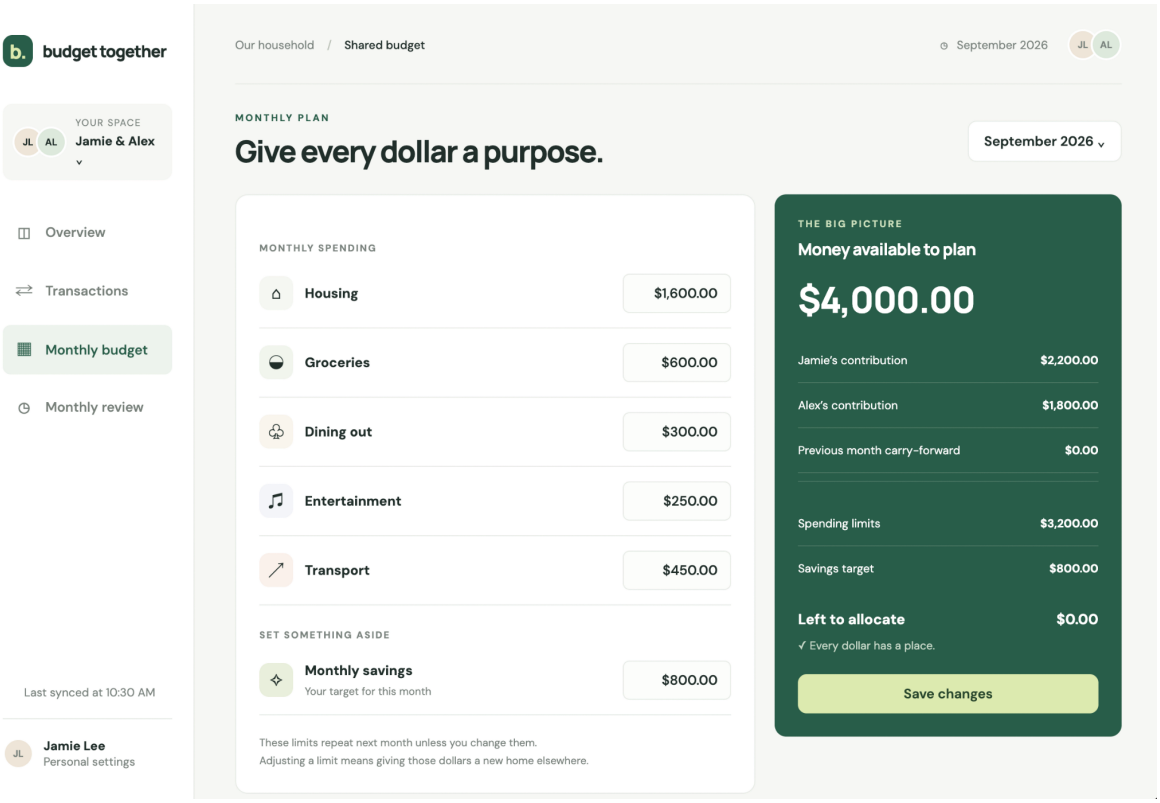


Transactions

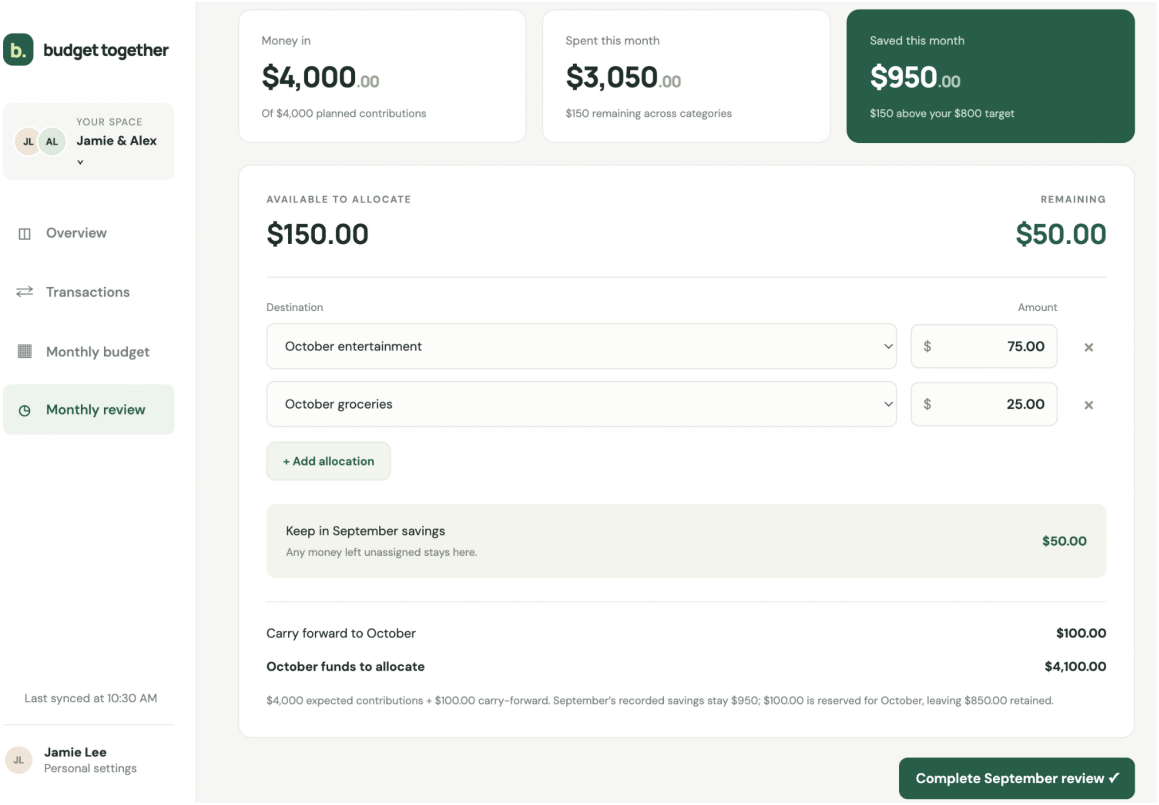


7. Views and wireframes (continued)

Budget



Monthly review



Accounts/settings: Connected accounts, private/shared assignments, connection status, last successful sync, partner invitation status, profile, disconnect controls, and leave-shared-budget action.

8. Initial data models

Conceptual models are independent of database or framework. IDs identify records and relationships.

Model	Core fields / purpose
User	Name, email, authentication identity, time zone, creation date.
BudgetSpace	Name, private/shared type, currency, reporting time zone.
Membership	User, budget, active/left status, join/leave dates; controls access.
Invitation	Shared budget, inviter, recipient email, status, expiration.
BankConnection	Connecting user, institution reference, status, last successful sync.
Account	Connection, budget, external reference, name, type, masked identifier, active status.
Category	Budget, name, order, optional color/icon.
MonthlyBudget	Budget, month, expected income, savings target, draft/active and review status.
CategoryAllocation	Monthly budget, category, spending limit.
SharedContribution	Private/shared budgets, month, planned amount, matched actual transfers.
Transaction	Account, source, external reference, description, amount, date, pending/posted status, classification, exclusion flag, note.
TransactionSplit	Transaction, category, amount; one allocation normally, several for splits.
CarryForwardAllocation	Source/destination months, destination category or savings target, amount.
RecurringBill	Budget, account, merchant, estimated amount, frequency, expected date, confirmation status.
Alert	Budget, category/month, type, creation date, per-user read state.

Relationships and constraints

At most two active members per shared budget and one active budget assignment per account. Categories and allocations must share a budget; splits must equal the transaction total. Syncs cannot duplicate transactions, and carry-forward cannot exceed available leftovers. Store money precisely to the cent; use US dollars and calendar months.

Categories and manual transactions satisfy the assignment's two-model CRUD requirement.

9. Proposed API endpoints

Route shorthand: **B = /budget-spaces/{id}**; **M = B/months/{month}**. Braces indicate route parameters.

Area	Methods and paths
Authentication	POST /auth/register, /auth/login, /auth/logout
Password recovery	POST /auth/password-reset-requests, /auth/password-resets
Profile	GET, PATCH /me; PUT /me/password
Budget spaces	GET, POST /budget-spaces; PATCH B
Invitations / membership	POST B/invitations; DELETE /invitations/{id}; POST /invitations/{token}/accept; POST B/leave
Bank linking / status	POST /bank-connections/link-sessions; GET, POST /bank-connections; DELETE /bank-connections/{id}
Reconnect / sync	POST /bank-connections/{id}/reconnect-sessions; POST /bank-connections/{id}/sync
Bank events	POST /bank-events (authenticated update events)
Accounts	GET /accounts; PATCH /accounts/{id} (name or budget assignment)
Categories	GET, POST B/categories; PATCH, DELETE /categories/{id}
Monthly budgets	GET, PUT M; POST M/activate (validate balance)
Contributions	GET B/contributions; PUT M/contributions
Transactions	GET, POST B/transactions; GET, PATCH, DELETE /transactions/{id} (delete manual entries only)
Splits	PUT /transactions/{id}/splits (validate total)
Dashboard	GET B/dashboard?month=YYYY-MM
Reports	GET B/reports?from=YYYY-MM&to=YYYY-MM
Monthly review	GET, PUT M/review (results and leftover allocations)
Recurring bills	GET B/recurring-bills; PATCH /recurring-bills/{id}
Alerts	GET B/alerts; PATCH /alerts/{id} (personal read status)
Live updates	B/events (authorized shared-budget live-update channel)

All endpoints enforce account and budget permissions. Transaction lists support pagination and filters. Invalid allocations return actionable errors.

10. Privacy, accuracy, and failure behavior

Privacy: Enforce private-data restrictions in both UI and direct requests. Show which history will be shared before confirming an account move. Leaving or unsharing preserves both partners' read access to previously shared records but stops future sharing from the departing person's accounts. Disconnecting stops sync while retaining history.

Sync: Deduplicate repeated imports and the same bank account connected by both partners. On failure, retain imported data, mark it stale, and offer reconnection. Show pending charges separately; posted transactions determine finalized totals and replace pending entries without duplication. Let users reconcile manual entries with matching imports.

Accounting: Refunds reduce category spending. Transfers are neither purchases nor earned income; shared contributions change each budget's funding without inflating combined income. Late or corrected transactions update the affected month and flag invalid carry-forward allocations.

Collaboration and accessibility: Conflicting edits prompt users to resolve updated values rather than silently overwrite changes. Support keyboard navigation, labeled controls, readable contrast, and text equivalents for charts.

11. Scope boundaries

Excluded: retirement/investment tracking, long-term savings balances, payments or money movement, bill splitting/debt tracking, groups larger than two, non-US banks, multiple currencies, native mobile apps, offline use, bill/monthly-review reminders, push notifications, and MVP subscription billing.

Bank coverage, connectivity cost, and transaction freshness depend on the connectivity provider; support for every US institution is not guaranteed. Technology selection and detailed implementation planning belong to the bakeoff unit.

12. Acceptance criteria

1. Registration and supported-bank linking produce automatic imports.
2. Selected accounts are shared while other accounts remain private; both partners can edit.
3. Activation rejects unbalanced plans; shared contributions balance both budgets consistently.
4. Categories and manual entries support CRUD; corrected and split transactions retain their totals.
5. Dashboard balances are accurate; transfers and card payments do not inflate income or spending.
6. Monthly review supports explicit carry-forward without automatic rollover or duplicate funds.
7. Shared edits appear promptly, and core tasks work on mobile and desktop browsers.

13. Team workflow and documentation

Use Git/GitHub and a project-management tool, with meaningful contributions from each member, weekly sprint goals, and brief planning/retrospective notes. Team membership and tracking-tool choice follow team formation.

The README covers project scope, team members, installation, configuration and bank connectivity, API documentation, known issues/limitations, and future improvements.

14. AI-use reflection

I used AI to interview me about users, budgeting rules, privacy, bank connections, and user experience. I made the product decisions: private/shared finances, equal partner permissions, automatic imports, zero-sum budgets, and explicit leftover allocation. I removed retirement tracking and reminders. AI helped clarify contributions and carry-forward, then organized my decisions into features, wireframes, models, endpoints, and acceptance criteria.